

《自然语言处理》实验课介绍

实验项目编号	实验项目名称	实验类型	实验性质	实验学时	每组人数	备注
1	NLP 开发环境与基础 文本处理	验证性	必做	2 学时	1	个人提交 实验报 告，实验 一、二合 并交一份 报告
2	中文分词与关键词提 取综合实验	综合性	必做	4 学时	1	
3	文本表示与文本分类 综合实验	综合性	必做	4 学时	1	个人提交 实验报告
4	课程设计：基于 ESP32 和语言大模型 的 AI 小智智能交互系 统设计	设计性	必做	8 学时	3	个人提交 课程设 计

总结：5 次实验课，提交 2 份实验报告，1 份课程设计报告

《自然语言处理》实验指导书

实验一 NLP 开发环境与基础文本处理

项目	内容
课程编号	HE00239
实验类型	验证性、必做
学时	2 学时（80 分钟）
组织形式	个人独立完成
实验环境	Python 3.10 及以上、VS Code 或 JupyterLab

一、实验目标

1. 掌握 Python 环境检查、脚本运行和文件路径的基本操作；
2. 掌握 UTF-8、UTF-8-SIG、GB18030 和 UTF-16 中文文本的读取方法；
3. 掌握 Unicode 规范化、空白处理、隐私字段掩码和基础词频统计；
4. 学会用中间输出和断言核验处理结果；
5. 能够记录“错误现象—原因—修改—复测”的调试过程。

二、实验文件

解压教师下发的学生资源包，目录应为：

```
实验一/  
├─ exp1_starter.py  
├─ stopwords.txt  
└─ data/  
    ├─ 任务表.csv  
    ├─ input_A_utf8.txt ... input_D_utf16.txt  
    └─ optional_E_utf8.txt ... optional_J_gb18030.txt
```

每位学生都要依次处理 A—D 四个必做文件，不再按学号分配。E—J 为选做题，可根据兴趣选择，选做不替代 A—D 任何一项。

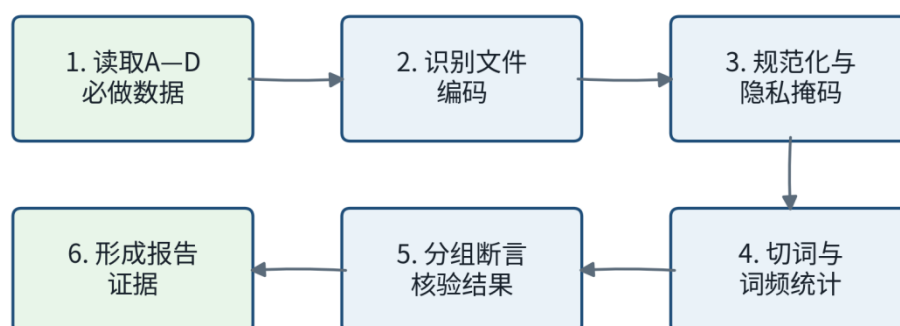
必做组	主题	文件编码	任务
A	健康数值与单位	UTF-8	保留血压、剂量单位、否定/约束词和段落
B	健康教育与隐私字段	UTF-8-SIG	掩码手机号和邮箱，保留血糖数值与单位
C	康复训练与全角字符	GB18030	规范全角字符，保留日期时间和检查术语

必做组	主题	文件编码	任务
D	检查记录与否定表达	UTF-16	停用词处理后保留无、未、不伴

选做组	主题	文件编码	任务
E	公共卫生宣教与段落	UTF-8	合并多余空白但保留两个语义段落
F	心电检查说明	GB18030	规范全角日期时间，保留检查术语
G	睡眠健康建议	UTF-8-SIG	只统一正文英文大小写，保留组别和数据编号
H	视力健康教育	UTF-16	过滤停用词时保留否定词“未”
I	用药安全科普	UTF-8	保留 5 mg/片与约束词“不得”
J	老年健康管理	GB18030	解释各阶段字符数变化和错误记录

实验一：基础文本处理与核验流程

编号表示执行顺序；每一步均保留输入、输出与核验依据



三、实验原理与程序实例

3.1 实例一：确认 Python 环境和当前目录

打开终端，依次执行：

```
python --version
python -c "import sys, pathlib; print(sys.executable); print(pathlib.Path.cwd())"
```

应看到 Python 版本、解释器路径和当前目录。若提示“找不到文件”，先用 `cd` 进入实验一目录，不要在代码中写其他同学电脑的绝对路径。

3.2 实例二：为什么要尝试多种编码

中文文本在磁盘保存为字节。用错误编码解释字节时会出现 `UnicodeDecodeError`。下面的函数依次尝试四种编码，成功后立即返回；失败信息保留在列表中。

```

from pathlib import Path

def read_text_checked(path):
    failed = []
    for encoding in ("utf-8", "utf-8-sig", "gb18030", "utf-16"):
        try:
            text = Path(path).read_text(encoding=encoding)
            return text, encoding, failed
        except UnicodeError as error:
            failed.append({"encoding": encoding,
                          "error": type(error).__name__})
    raise UnicodeError(f"无法识别文件编码: {path}")

```

以 C 组 GB18030 文件为例：

```

text, encoding, failed = read_text_checked("data/input_C_gb18030.txt")
print("识别编码: ", encoding)
print("失败记录: ", failed)
print(text.splitlines()[0])

```

实际输出：

```

识别编码:  gb18030
失败记录:  [{'encoding': 'utf-8', 'error': 'UnicodeDecodeError'},
            {'encoding': 'utf-8-sig', 'error': 'UnicodeDecodeError'}]
【任务数据 C：康复训练与全角字符】

```

这里的两次失败是有价值的诊断证据。不得使用 `errors="ignore"` 跳过错误，因为它会静默丢失字符。

3.3 实例三：Unicode 规范化

C 组文件中的“3 0”是全角数字。NFKC 规范化后变为“30”。

```

import unicodedata

before = "上肢抬举 3 0 次"
after = unicodedata.normalize("NFKC", before)
print(before)
print(after)

```

输出：

```

上肢抬举 3 0 次
上肢抬举 30 次

```

规范化后必须检查医学信息是否仍然保留。不能用“只保留汉字”的正则清洗，因为它会删除 128/78 mmHg、5 mg/片等有意义内容。

3.4 实例四：隐私字段掩码

```
import re

def mask_privacy(text):
    text, mobile_n = re.subn(
        r"(?!\\d)(1[3-9]\\d)\\d{4}(?!\\d)",
        r"1****2", text)
    text, email_n = re.subn(
        r"([A-Za-z0-9._%+-]{2})[A-Za-z0-9._%+-]*(@[A-Za-z0-9.-]+)",
        r"1***2", text)
    return text, {"mobile": mobile_n, "email": email_n}

sample = "电话 13912345671，邮箱 student1@example.com"
print(mask_privacy(sample))
```

输出：

```
('电话 139****5671，邮箱 st***@example.com', {'mobile': 1, 'email': 1})
```

掩码只用于本实验中的合成数据。它不是经过临床验证的完整脱敏系统。

3.5 实例五：什么是断言

断言（assertion）是程序中的“自动检查题”。程序员先写出对结果的预期：条件为 True 时，程序继续运行；条件为 False 时，Python 立即报出 AssertionError，告诉我们某项结果与预期不一致。

基本语法是：

```
assert 需要检查的条件, "条件不成立时显示的提示"
```

例如，否定词“未”不能在清洗中被删除：

```
clean_text = "患者未见异常放电。"
assert "未" in clean_text, "清洗后丢失否定词‘未’"
print("断言通过：否定词已保留")
```

正常输出：

```
断言通过：否定词已保留
```

现在故意删除“未”，观察断言如何定位问题：

```
wrong_text = clean_text.replace("未", "")
assert "未" in wrong_text, "清洗后丢失否定词‘未’"
```

运行后将看到：

```
AssertionError: 清洗后丢失否定词‘未’
```

这不是要隐藏的错误，而是断言成功发现了语义丢失。学生应根据提示修改清洗规则，再次运行，直到断言通过。

3.5.1 本实验要写的6条断言

下面给出可直接模仿的六类断言。`clean_A`、`privacy_B`、`clean_C`和`tokens_D`分别代表A—D四组的处理结果；学生应按自己程序中的变量名调整。

```
# 1. 通用检查：输入文件必须存在
assert input_path.is_file(), f"输入文件不存在：{input_path}"

# 2. 通用检查：解码后不应有 Unicode 替换字符
assert "\ufffd" not in raw_text, "文本中出现替换字符，可能使用了错误编码"

# 3. A 组专门断言：数值、单位、否定词和段落都保留
assert all(x in clean_A for x in
           ("128/78 mmHg", "5 mg/片", "无", "不伴", "不得", "\n\n")), "A 组语义信息丢失"

# 4. B 组专门断言：手机号和邮箱应各命中 1 处
assert privacy_B["mobile"] == 1 and privacy_B["email"] == 1, "B 组隐私字段命中数不正确"

# 5. C 组专门断言：全角数字已转换，日期时间仍保留
assert "3 0 " not in clean_C and "2026-10-15 09:30" in clean_C, "C 组全角规范化失败"

# 6. D 组专门断言：关键否定表达均保留
assert all(x in tokens_D for x in ("无", "未", "不伴")), "D 组否定表达丢失"
```

一条 `assert` 语句计为一条断言，即使其条件中同时检查多个值。本实验中，共通过六条及以上，且 A、B、C、D 每组至少有一条专门断言，即满足要求。

3.5.2 断言与 `if` 判断的区别

- `assert` 用于检查“正常情况下必须成立”的内部条件，适合实验测试和调试；
- `if` 用于处理预期中可能发生的不同业务分支，例如“是否检出手机号”；

• Python 以优化模式 `python -O` 运行时可以跳过断言，因此不应用 `assert` 代替正式的用户输入校验或医疗安全判断。

课堂操作时，先预测断言应该通过还是失败，再运行程序。如果失败，按“错误提示—检查变量—修改规则—再次运行”的顺序完成调试。

3.6 实例六：词频统计和结果核验

```
import re
from collections import Counter

def tokenize(text):
    pattern = r"患者|建议|进行|不伴|否认|不得|无|未|[A-Za-z]+\d+(?:\.\d+)?|[\u4e00-\u9fff]"
    return re.findall(pattern, text)

tokens = tokenize("患者无发热，未见皮疹，不伴胸痛。")
freq = Counter(tokens)
print(tokens)
assert "无" in tokens
assert "未" in tokens
assert "不伴" in tokens
print("3 assertions passed")
```

输出：

```
['患者', '无', '发', '热', '未', '见', '皮', '疹', '不伴', '胸', '痛']
3 assertions passed
```

把“不伴”写在单汉字规则之前，是为了优先匹配完整否定短语。

四、上机实验步骤

步骤 1 核对四个必做文件（10 分钟）

1. 确认 `input_A`、`input_B`、`input_C`、`input_D` 四个文件均存在；
2. 不修改原始文件，输出写入独立目录；
3. 在报告中列出 A—D 四个文件名及任务。

步骤 2 完成编码识别（10 分钟）

把实例二的 `read_text_checked` 写入 `exp1_starter.py`，输出：实际编码、失败候选、第一行文本。UTF-8-SIG 文件可能由 `utf-8` 成功读取但首字符含 BOM，应检查并处理。

步骤 3 完成规范化和空白处理（10 分钟）

```
def normalize_text(text):  
    text = unicodedata.normalize("NFKC", text)  
    text = text.replace("\r\n", "\n").replace("\r", "\n")  
    text = re.sub(r"[\t\u3000 ]+", " ", text)  
    text = re.sub(r"\n{3,}", "\n\n", text)  
    return text.strip()
```

在报告中给出一处“规范化前—规范化后”对比；没有明显变化时说明原因，不得伪造变化。

步骤 4 完成隐私掩码和四组必做任务（15 分钟）

每人用同一处理流水线依次运行 A—D：A 核验数值/单位/段落，B 核验手机号和邮箱掩码，C 核验全角规范化与日期时间，D 核验否定词保护。每组写一条专门断言。

步骤 5 切词、停用词和词频（15 分钟）

1. 调用 `tokenize`;
2. 从 `stopwords.txt` 读取停用词;
3. “无、未、不伴、不得”不得作为停用词删除;
4. 输出 top-10 的词。

步骤 6 现场核验与整理报告（20 分钟）

教师按“班级一名单序号”发放一张现场新句任务卡。两个班各有 30 张不同任务卡。学生先在报告上写出预期处理结果，再运行程序，并解释一条与任务卡要求对应的断言。把预测与实际不一致之处写入调试记录。

五、上机实验题目

1. 完成 `exp1_starter.py` 中的全部 TODO，使程序可用一条命令运行;
2. 对 A—D 四个必做文件逐一完成编码识别、规范化、隐私检查、切词和词频;
3. 完成 A—D 表中的四项专项任务;
4. 编写 4 条断言，其中 A、B、C、D 每组有 1 条专门断言;
5. 记录至少 1 个真实调试过程;

6. 选做：从 E—J 中选择一组或多组，对比与 A—D 的处理差异；
7. 回答思考题并完成个人实验报告。

六、思考题

1. 为什么不能把所有数字、英文和标点全部删除？结合个人数据举例。
2. `errors="ignore"`可能造成什么问题？
3. NFKC 规范化可能带来哪些收益和风险？
4. 隐私正则可能产生哪些漏报或误报？

七、实验报告与提交要求

报告总页数不超过 10 页，封面、关键代码、截图和附录均计入页数。完整程序保留在本人电脑，供课堂抽查。

报告必须包含：实验目的；实验原理；实验仪器及实验环境；A—D 四个数据文件及结果摘要；处理流程；带中文注释的关键代码片段；规范化前后对比；至少 1 条“报错现象—原因定位—修改—复测”记录；实验总结与感悟；思考题；AI 工具使用说明（AI 说明写明工具/模型、使用目的、关键提问概述、采用内容及人工核验；未使用 AI 时明确写“未使用”）。选做 E—J 的结果放在“选做”小节，与必做结果分开；本人电子签名。